

Mira VPN

Server Specification

v1.0 · June 2026 · Vualet · Dubai

Confidential — Hetzner nbg1 · 178.104.251.30

0. Mira VPN — Brand Context

Mira is an **AI-assisted VPN**. She measures latency to nearby Mira servers, picks the fastest one on every connect, encrypts the connection, and gets out of the way. No country picker. No settings file. No guesswork.

Four principles: **She measures. She picks. She keeps you private. She pauses for you.** (One toggle for banking apps; per-app Direct list for permanent exclusions.)

Mira VPN is one product in the **Vualet** family (AI tools out of Dubai by Dr. Abdolmadjid Masoomi: Mira assistant, Veridian telephony, aFAQ OS, Primaion research). Public face: `vpn.mira.vualet.com` (DNS pending; live at `178.104.251.30`).

1. Objective

Define the server infrastructure to run Mira VPN at production scale — thousands of concurrent users, clear tier segmentation (free vs paid), automated abuse safeguards, infrastructure that feels fast regardless of tier. Target: Hetzner cx33 at `178.104.251.30` → future nodes behind Hetzner LB.

2. Live State

Server	mira-vpn-01 · Hetzner cx33 · 4 cores 8 GB 80 GB SSD 22 TB
OS	Ubuntu 24.04 · kernel 6.8.0-111
IP	178.104.251.30
Disk	2.9 / 75 GB (4%)
RAM	665 / 7800 MB (9%)
WireGuard	wg0 · 10.66.66.0/24 · pubkey ByK5yGC96gYi0GZ/dloyVBWZkLrCGyQH+XKtxS1UJhE= · listen 51820 · 0 peers
Services	mira-web :3060 (100M) · mira-api :5103 (31M) · x-ui :7091 (95M) · xray :8443 (46M) · nginx :80 (10M)
Firewall	ufw active · 22 80 443 8443 51820/udp open
tc	NOT APPLIED — clean system

3. Tier Control — Traffic Shaping

3.1 Tier Matrix

	Free	Paid (\$9.99/mo)
Speed per IP	500 kbps	5 Mbps
Volume	unlimited (speed-capped)	50 GB / month
Session	2 h, then drop	unlimited
Concurrent IPs	1	3
Server	nearest only	smart routing

3.2 tc htb (Initial)

```
#!/bin/bash
# /opt/mira-vpn/tc-tiers.sh - idempotent, run at boot
IFACE=enp1s0 PORT=51820
tc qdisc del dev $IFACE root 2>/dev/null || true
tc qdisc add dev $IFACE root handle 1: htb default 99
tc class add dev $IFACE parent 1: classid 1:1 htb rate 1000mbit ceil 1000mbit
tc class add dev $IFACE parent 1:1 classid 1:99 htb rate 200mbit ceil 1000mbit # system
tc class add dev $IFACE parent 1:1 classid 1:2 htb rate 800mbit ceil 800mbit # vpn pool
tc class add dev $IFACE parent 1:2 classid 1:10 htb rate 5mbit ceil 5mbit # paid per IP
tc class add dev $IFACE parent 1:2 classid 1:20 htb rate 500kbit ceil 500kbit # free per IP
tc filter add dev $IFACE protocol ip parent 1:0 prio 99 u32 match ip sport $PORT 0xffff flowid 1:20
```

3.3 Per-IP MARK (Production)

```
# Inside tunnel-api.py add_peer():
def apply_mark(ip, tier):
    mark = "0x10" if tier == "paid" else "0x20"
    subprocess.run(["iptables", "-t", "mangle", "-A", "POSTROUTING", "-s", ip, "-j", "MARK", "--set-mark", mark], check=True)
```

Then tc filters by mark: `match mark 0x10 0xff` → paid class, `0x20` → free.

3.4 2-Hour Free Session Prune

```
#!/bin/bash
NOW=$(date +%s) MAX=7200
wg show wg0 dump | tail -n+2 | while read pubkey _ ip _ _ _ handshake _; do
    [ -z "$pubkey" ] && continue
    AGE=$((NOW-handshake))
    if iptables -t mangle -L POSTROUTING -n | grep -q "$ip.*0x20" && [ $AGE -gt $MAX ]; then
        wg set wg0 peer "$pubkey" remove
        iptables -t mangle -D POSTROUTING -s "$ip" -j MARK 2>/dev/null
        echo "pruned free peer $ip (${AGE}s)"
    fi
done
```

Systemd timer: `/etc/systemd/system/mira-prune.timer` · OnCalendar `*:0/5` (every 5 min).

3.5 Volume Cap Enforcement

Parsed from `wg show wg0 transfer` every prune cycle. If per-peer received+transmitted exceeds 50 GB for paid tier, remove peer. Accumulate in SQLite `peers.db`.

4. Nginx Rewrite & Acceleration

4.1 Gzip + Cache Headers

```
gzip on; gzip_types text/css application/javascript image/svg+xml application/json; gzip_min_length 1000;
location /_next/static/ { expires 365d; add_header Cache-Control "public,immutable"; }
location /mira/ { expires 7d; add_header Cache-Control "public"; }
```

4.2 API Rate Limiting

```
# nginx.conf http{}
limit_req_zone $binary_remote_addr zone=mira_api:10m rate=5r/s;
location /v1/ { limit_req zone=mira_api burst=10 nodelay; ... }
```

4.3 Timeout Tuning

```
location /v1/ { proxy_buffering on; proxy_read_timeout 30s; proxy_connect_timeout 5s; }
location / { proxy_read_timeout 60s; proxy_connect_timeout 10s; }
```

5. Kernel Tuning for Speed Perception

```
# /etc/sysctl.d/90-mira-vpn.conf
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
net.ipv4.tcp_keepalive_time = 60
net.ipv4.tcp_keepalive_intvl = 10
net.ipv4.tcp_keepalive_probes = 6
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_fastopen = 3
fs.file-max = 2097152
```

6. DNS Acceleration

```
apt-get install -y unbound
# /etc/unbound/unbound.conf.d/mira.conf
server:
    interface: 10.66.66.1
    port: 53
    access-control: 10.66.66.0/24 allow
    prefetch: yes
    cache-min-ttl: 3600
    serve-expired: yes
    qname-minimisation: yes
forward-zone: name: "." forward-addr: 1.1.1.1 forward-addr: 1.0.0.1
systemctl enable --now unbound
```

Then update WireGuard DNS to `10.66.66.1` in the API config response. Cached DNS = 0 ms for hotspot queries.

7. Abuse Safeguards

7.1 Per-Peer Connection Limits

```
iptables -I FORWARD -i wg0 -p tcp -m connlimit --connlimit-above 50 --connlimit-mask 32 -j REJECT --reject-with tcp-reset
iptables -I FORWARD -i wg0 -p udp -m connlimit --connlimit-above 200 --connlimit-mask 32 -j DROP
```

7.2 Registration Rate Cap (Python)

```
_registry = defaultdict(list)
PEER_WINDOW = 3600; MAX = 5
def rate_check(ip):
    now=time.time(); _registry[ip]=[t for t in _registry[ip] if now-t<MAX: return False
    _registry[ip].append(now); return True
```

Return 429 Too Many Requests if exceeded.

7.3 Block P2P on Free Tier

```
iptables -A FORWARD -i wg0 -m mark --mark 0x20 -p tcp -m multiport --dports 6881:6889,6969,1337,51413 -j REJECT --reject-with
```

7.4 Anti-Spoofing

```
iptables -A FORWARD -i wg0 -s 10.66.66.0/24 -j ACCEPT
iptables -A FORWARD -i wg0 -j DROP
```

8. Peer Database (SQLite)

```
# In tunnel-api.py
conn = sqlite3.connect('/opt/mira-api/peers.db')
conn.execute("CREATE TABLE IF NOT EXISTS peers(pubkey TEXT PRIMARY KEY, ip TEXT, tier TEXT DEFAULT 'free', registered_at TEXT,
```

9. Process Watchdog

```
# /opt/mira-vpn/watchdog.py
while True:
    for svc in ["mira-web", "mira-api", "nginx"]:
        if subprocess.run(["systemctl", "is-active", svc]).returncode:
            subprocess.run(["systemctl", "restart", svc])
    time.sleep(60)
```

Systemd unit: `/etc/systemd/system/mira-watchdog.service`

10. Impact Summary

Before	After
No shaping — one user saturates 143 Mbps	tc per-tier: free 500k, paid 5M, system 200M
No peer database	SQLite peers.db tracking every issued peer
No session limit	2-hour free prune timer
No volume cap	iptables + wg transfer + 50 GB check
No API rate limiting	5 r/s per IP at nginx
No conn limits	50 TCP / 200 UDP per peer
No DNS cache	unbound local resolver, 0 ms cache hit
No BBR / TCP tuning	BBR + fq + buffer sizing
No process watchdog	systemd timer auto-restarts dead services
No anti-spoofing	iptables limits forwarding to our subnet only

11. Build Order

1. Kernel tuning (BBR, buffers, TCP fast open) — 5 min · highest impact
2. tc tier shaping — 10 min
3. nginx rate limiting + gzip + caching — 10 min
4. Peer DB + wire into tunnel-api.py — 15 min
5. 2-hour free session prune timer — 10 min
6. iptables MARK system (per-IP shaping) — 15 min
7. unbound DNS — 5 min
8. Watchdog — 10 min
9. Connlimit + torrent + subnet filter — 5 min
10. Volume cap check — 10 min

Total under 2 hours for the full stack.